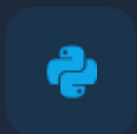


System e-commerce sklepu odzieżowego

Część 5: Testy systemu i zapewnienie jakości (QA)

Kacper Gumulak & Damian Judka

5.1 Metodologia i środowisko testowe



Python & Pytest

Zautomatyzowany framework testowy napisany w języku Python 3.12 z wykorzystaniem biblioteki **pytest**.



Izolacja Sesji

Każdy test działa w ramach dedykowanej, odizolowanej sesji. Zapobiega to przypadkowej ingerencji w dane produkcyjne.



MariaDB 11

Bezpośrednie testowanie logiki rdzenia bazy danych w oparciu o silnik InnoDB, omijając narzut aplikacji frontendowej.



PyMySQL

Lekki, natywny sterownik użyty do komunikacji na linii skrypty testowe Python - kontener bazy danych.

Klasyfikacja przypadków testowych



Jednostkowe (Unit)

4 przypadki

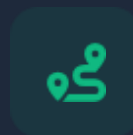
Weryfikacja najbardziej podstawowych mechanizmów: sprawdzanie unikalności danych, testowanie algorytmów generowania haseł oraz walidacja ilościowa w inwentarzu.



Modułowe (Integration)

2 przypadki

Badanie współpracy konkretnych procesów wewnątrz bazy. Obejmuje m.in. odrzucanie pustych koszyków oraz weryfikację blokady mechanizmu oversellingu.



Funkcjonalne (E2E)

2 przypadki

Symulacja pełnej ścieżki życia obiektu. Od założenia koszyka, przez zakup, symulację płatności i faktury, aż po zwrot towaru weryfikujący moduł Event Sourcing.

Diagnoza 1: Błąd zarządzania transakcjami

Wykryty Problem

Konflikt podczas testów integracyjnych na linii sterownik-procedura. Próba zmiany poziomu izolacji z poziomu kodu SQL zawiodła, ponieważ aplikacja kliencka (Python) zainicjowała już własną transakcję.

```
E pymysql.err.OperationalError: (1644,  
"Transaction characteristics can't be changed  
while a transaction is in progress")
```

Rozwiązanie

Uznano, że sterowanie ręczne transakcjami z poziomu skryptu testowego zakłóca architekturę. Wprowadzono przełączenie sterownika PyMySQL w tryb **autocommit**.

Dzięki temu pełna kontrola nad transakcyjnością i poziomami izolacji została prawidłowo oddelegowana bezpośrednio do silnika bazy danych.

Diagnoza 2: Luka w integralności referencyjnej

Wykryty Problem

Testy jednostkowe "wysypały się" w momencie próby usunięcia testowego konta użytkownika. Relacyjna baza danych słusznie zablokowała operację ze względu na istniejący koszyk powiązany z tym kontem.

```
E pymysql.err.IntegrityError: (1451, "Cannot delete or update a parent row: a foreign key constraint fails (`cart`, CONSTRAINT `customer_fk2`)" )
```

Rozwiązanie

Zidentyfikowano brak strategii kaskadowego usuwania w modelu danych. Wdrożono poprawkę architektoniczną w schemacie tabeli koszyka.

```
ALTER TABLE `cart` ADD CONSTRAINT `customer_fk2`  
FOREIGN KEY (`customer_id`) REFERENCES `customers`  
(`customer_id`) ON DELETE CASCADE;
```

Diagnoza 3: Konflikt kolacji znaków (Collation)

Wykryty Problem

Środowisko wirtualne obnażyło błąd porównywania ciągów tekstowych. Baza danych nie potrafiła wykonać operacji logicznych ze względu na miks dwóch różnych standardów kodowania znaków dla poszczególnych tabel.

```
E OperationalError: (1644, "Illegal mix of
collations (utf8mb4_general_ci) and
(utf8mb4_uca1400_ai_ci) for operation '='")
```

Rozwiązanie

Aby zapobiec takim problemom u innych deweloperów po pobraniu repozytorium, ujednolicono standard kodowania na poziomie samego kontenera.

Dodano poniższe parametry startowe do pliku konfiguracyjnego **docker-compose.yml**:

```
command: - --character-set-server=utf8mb4 - --
collation-server=utf8mb4_general_ci
```

Podsumowanie prac QA

8/8
PASSED

System w pełni operacyjny

Po wdrożeniu powyższych poprawek architektonicznych, wszystkie zdefiniowane scenariusze testowe przechodzą bezbłędnie.

Szczególnym sukcesem jest pozytywna validacja mechanizmu **Event Sourcing**. System niezawodnie przelicza dostępność asortymentu w czasie rzeczywistym, nawet w skomplikowanych scenariuszach anulowania opłaconych już zamówień.

Dziękujemy za uwagę

Zakończyliśmy omawianie fazy testowania i zapewniania jakości.

Jesteśmy do dyspozycji w razie ewentualnych pytań.